엔터프라이즈 권한관리 및 인가 서비스 설계서

문서 버전: 2.3 **작성일**: 2026-04-25 **대상 시스템**: 다중 시스템 통합 권한관리 플랫폼

목차

1. 설계 철학 및 방침
2. 핵심 개념 및 용어
3. 요구사항 및 규모
4. 전체 아키텍처
5. 데이터 모델
6. 캐시 전략 (샤딩 키 구조)
7. 권한 조회(인가) 서비스
8. 권한 변경 및 동기화
9. 이벤트 기반 Warm-up
10. 관리 화면 UI 설계
11. 성능 목표 및 검증
12. 운영 및 감사
13. 구현 로드맵

---

1. 설계 철학 및 방침

1.1 기본 철학

- **단순함 우선**: 복잡한 규칙보다 단순한 원칙을 일관되게 적용
- **조회 경로 최적화**: 5ms / 1,000 TPS 보장을 최우선 설계 목표로 둠
- **변경 비용과 조회 비용의 균형**: 변경은 느려도 되나 조회는 반드시 빨라야 함
- **의미 보존**: 권한 출처(어느 경로로 부여되었는지)를 언제든 추적 가능해야 함
- **경계 명확화**: 시스템/회사 단위로 권한 경계를 분명히 분리
- **L1/L2 키 일관성**: Caffeine과 Redis의 키 구조를 통일하여 무효화·운영 단순화
- **명시적 컨피그**: 추론·매칭 규칙보다 명시적 위치 지정을 우선
- **메뉴 라이프사이클 통합**: 메뉴 정의·구현체 메타·액션·API 매핑을 한 곳에서

1.2 핵심 방침

방침	내용
Allow-only	명시적 허용만 관리. Deny 권한은 도입하지 않음
무조건 상속	Company → 소속 User 전체에 자동 전파 (옵션 없음)
그룹 지원	Company/Dept/User 각 레벨에 그룹 개념 제공
시스템 경계	모든 권한은 `system_cd` 기준으로 분리 관리
시스템 메타	시스템은 코드뿐 아니라 소유 회사·사업부·책임자·기술 스택 메타를 보유
메뉴 트리	시스템 하위에 `Folder/Menu/Link` 단일 트리 (메뉴그룹 별도 개념 제거)
메뉴 구현체 메타	라우트·컴포넌트 정보를 별도 테이블로 분리 관리
메뉴-액션-API 체계	권한은 (메뉴, 액션) 단위, 폴더 부여 시 자손 리프로 자동 전개
샤딩 키 구조	(method, depth) 기반 샤드로 단일 Set 크기 축소
L1/L2 키 통일	Caffeine과 Redis 모두 동일 key 구조, 값은 Set
Eventual Consistency	권한 변경은 DB 즉시, 캐시는 수 초 내 반영
Defense in Depth	3계층 캐시(Caffeine/Redis/DB)로 성능 및 가용성 확보

1.3 금지 원칙

- 그룹의 중첩 금지 (UG 안에 UG, DG 안에 DG 등 재귀 구조 불허)
- Deny 권한 도입 금지
- 조직 계층 구조 도입 금지 (부서는 flat)
- 조회 경로에 DB 쿼리 상시 포함 금지
- 폴더 메뉴에 액션-API 매핑 금지 (리프만 허용)

---

2. 핵심 개념 및 용어

2.1 시스템 (System)

권한 경계의 최상위 단위. 단순 코드가 아닌 풍부한 메타정보를 보유:

메타 영역	항목
식별	system_cd, system_nm (한/영), description
소유·책임	owner_company_cd, owner_division, owner_dept_id, owner_user_id
분류	system_type (ERP/MES/PLM/PORTAL/HR/ETC), system_category
기술 메타	base_url, frontend_type (VUE/REACT/JSP/...)
운영 메타	status, go_live_date, end_of_life_date
추가 속성	TB_SYSTEM_ATTR (key-value, 자유 확장)

2.2 메뉴 (Menu)

시스템 하위의 **단일 트리 구조**. 모든 노드는 동일 테이블에 저장되며 `menu_type` 으로 역할 구분:

menu_type	명칭	역할	액션-API 매핑	구현체 메타
F	Folder	하위 메뉴를 묶는 중간 노드	✗ 없음	✗ 없음
M	Menu	실제 화면 (리프)	✓ 가능	✓ 보유
L	Link	외부 링크 (다른 시스템 등)	✗ 없음	✓ 외부 URL

핵심 변화 (v2.2 이전 대비): - `TB_MENU_GROUP` 별도 개념 제거 - 메뉴 트리의 폴더(F)가 그룹 역할 수행 - 폴더에 권한 부여 시 자손 리프(M)로 flatten 단계에서 자동 전개

2.3 메뉴 구현체 메타 (Menu Implementation)

리프 메뉴(M)와 링크(L)는 화면 구현체와 연결됨. 별도 테이블 `TB_MENU_IMPL` 에 분리:

```
Vue Router 정보:
  - route_path, route_name, component_name, component_path
Route Meta (JSON):
  - { requiresAuth, layout, keepAlive, title, ... }
Route 파라미터/쿼리:
  - params, query (JSON)
화면 옵션:
  - has_layout, is_full_screen, is_modal, mobile_supported, mobile_route_path
외부 링크 (Link 유형):
  - external_url, open_target
```

2.4 주체(Subject) 유형

코드	명칭	설명
C	Company	회사
CG	CompanyGroup	회사 묶음
D	Department	부서 (회사 내 flat)
DG	DeptGroup	부서 묶음
U	User	개인 사용자
UG	UserGroup	사용자 묶음

2.5 대상(Target) 유형

코드	명칭	설명
M	Menu (트리 노드)	폴더(F) 또는 리프(M). 폴더 부여 시 자손 리프로 전개

v2.2 이전과의 차이: `target_type='G'` (MenuGroup) 제거. 트리 자체가 그룹 역할.

2.6 액션(Action)

코드	의미	비고
R	Read (조회)	기본
C	Create (생성)	
U	Update (수정)	
D	Delete (삭제)	
A	Approve (승인)	확장
X	Export (내보내기)	확장
P	Print (출력)	확장
...	시스템별 정의	자유롭게 확장

2.7 상속 규칙

```
Company 권한 → 해당 회사의 모든 User에 자동 상속
Dept    권한 → 해당 Dept 소속 User에 적용 (Dept 간 상속 없음)
Folder  권한 → 자손 리프 메뉴 전체로 자동 전개 (메뉴 트리 상속)
Group   권한 → 멤버 전원에 적용
```

2.8 권한 결정 공식

```
allowed(user, api) =
  (user 소속 Company의 api 권한)
  ∪ (user 소속 Dept의 api 권한)
  ∪ (user 개인/그룹 경유 api 권한)

any one path = allow
```

API 권한 계산 시점에 폴더 → 자손 리프 → 액션-API 매핑이 모두 풀린 상태로 캐시에 적재됨.

2.9 샤딩 관련 용어

용어	설명
샤드 키	권한 Set을 분할하는 보조 키 차원
method	HTTP 메서드(GET/POST/PUT/DELETE/PATCH)
depth	URL 패턴의 디렉토리 세그먼트 수
SEG (segment)	URL의 특정 위치 세그먼트 (시스템별 컨피그 위치 지정)
segment_position	0-indexed, URL 분할 후 추출할 세그먼트 인덱스

---

3. 요구사항 및 규모

3.1 기능 요구사항

- 시스템별 메뉴/API 등록 관리 (메뉴는 트리 + 구현체 메타 포함)
- 메뉴별 CRUD 및 확장 액션 권한 관리
- 회사/부서/사용자 및 그룹 단위 권한 부여/해제
- 메뉴 접근 권한 조회 API
- API 호출 인가 체크 API
- 권한 변경 시 실시간 캐시 반영
- 권한 출처 추적 및 감사 로그

3.2 비기능 요구사항

항목	목표
조회 응답 시간	5ms 이하 (P99 포함)
조회 처리량	1,000 TPS
권한 변경 반영 지연	30초 이내 (일반), 즉시 (단일 변경)
가용성	99.9% 이상

3.3 데이터 규모

항목	규모
시스템	다수 (ERP, MES, PLM, 포털 등)
회사	10개
부서	회사당 평균 300개 (총 3,000개)
사용자	전체 10만명
메뉴 (전체)	10만개 (폴더 + 리프 + 링크)
메뉴 (리프만, 권한 부여 의미 있는 단위)	~7만개 추정
API	메뉴당 평균 5개 → 총 약 50만개
사용자당 권한 API	평균 500개

4. 전체 아키텍처

4.1 논리 구성도

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                    Client / BFF                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                          ┌───────┴────────┐
                          │                │
                   [관리 화면]        [인가 조회]
                          │                │
┌──────────────────────────▼──────────┐   ┌──────────────────▼──────────────────────────────────────────┐
│ 권한관리 서비스                      │   │ 권한조회(인가) 서비스                                       │
│ - 시스템·메뉴 마스터                 │   │ - API 권한 체크                                             │
│ - 권한 부여/해제                     │   │ - 메뉴 권한 조회                                            │
│ - 그룹 관리                          │   │ - Warm-up 처리                                              │
└──────────────────────────────────────┘   └─────────────────────────────────────────────────────────────┘
        │                                │
        │                                │
        │                                │
        ▼                                ▼
┌──────────────────────────────┐   ┌──────────────────────┐   ┌──────────────────────┐
│ PostgreSQL                   │   │ Caffeine L1          │   │ Redis L2             │
│ (Source of                   │◀──Sync──▶│ (서버별,             │◀──────────▶│ (공유,               │
│  Truth)                      │   Worker │  동일 key)           │ Pub Sub │  동일 key)           │
└──────────────────────────────┘   └──────────────────────┘   └──────────────────────┘
        │
        │ Outbox
        ▼
┌──────────────────────────────┐
│ Kafka                        │  ← 대규모 변경 비동기 처리
│ (perm events)                │
└──────────────────────────────┘
```

4.2 조회 플로우

```
Request (system, company, dept, user, menu, method, api_url)
   ↓
1. URL → (api_id, depth, seg) 메타 해석
   ↓
2. 시스템 샤딩 전략 기반 샤드 결정
   shard = "{method}:{depth}"           (METHOD_DEPTH)
       또는 "{method}:{depth}:{seg}"   (METHOD_DEPTH_SEG)
   ↓
3. 3개 샤드 키:
   k1 = perm:api:{s}:C:{c}:{shard}
   k2 = perm:api:{s}:D:{c}:{d}:{shard}
   k3 = perm:api:{s}:U:{u}:{shard}
   ↓
4. for each k:
     Caffeine.get(k, k -> redis.smembers(k))
     if api_id in Set: return TRUE
   return FALSE
```

4.3 변경 플로우

```
관리 화면에서 권한 변경 (또는 메뉴 트리 변경)
   ↓
권한관리 서비스
   - TB_PERMISSION 또는 TB_MENU 갱신
   - TB_PERM_CHANGE_LOG INSERT (동일 TX)
   ↓
Sync Worker (Polling 또는 Debezium CDC)
  - 영향 캐시 레벨/샤드 판단
  - 폴더 → 자손 리프 재귀 전개 (필요 시)
   - Redis Set rebuild (Pipeline) — 영향 샤드만
   - Pub/Sub publish → Caffeine 무효화
   - TB_PERM_CHANGE_LOG.processed_yn = 'Y'
```

---

5. 데이터 모델

5.1 시스템 마스터

```
-- 시스템 (메타정보 강화)
TB_SYSTEM (
  system_cd       VARCHAR(20)  PK,
  system_nm       VARCHAR(100) NOT NULL,
  system_nm_en     VARCHAR(100),
  description      VARCHAR(500),

  -- 소유/책임 메타
  owner_company_cd VARCHAR(10),              -- FK TB_COMPANY
  owner_division   VARCHAR(50),              -- 사업부 (조선사업부 등)
  owner_dept_id    VARCHAR(50),              -- 운영 부서
  owner_user_id    VARCHAR(50),              -- 시스템 책임자

  -- 분류
  system_type      VARCHAR(20),              -- ERP/MES/PLM/PORTAL/HR/ETC
  system_category  VARCHAR(50),              -- 비즈니스 도메인

  -- 기술 메타
  base_url        VARCHAR(500),              -- https://erp.hd.co.kr
  frontend_type    VARCHAR(20),              -- VUE/REACT/JSP/THYMELEAF/ETC

  -- 운영 메타
  status          CHAR(1),                   -- A:활성, P:준비, I:비활성, D:폐기
  go_live_date     DATE,
  end_of_life_date DATE,

  created_at      TIMESTAMP,
  updated_at      TIMESTAMP,
  updated_by      VARCHAR(50)
);
CREATE INDEX idx_system_owner_company ON TB_SYSTEM(owner_company_cd);
CREATE INDEX idx_system_status       ON TB_SYSTEM(status);

-- 시스템 추가 속성 (자유 확장)
TB_SYSTEM_ATTR (
  system_cd       VARCHAR(20),
  attr_key        VARCHAR(50),
  attr_value      VARCHAR(500),
  PRIMARY KEY (system_cd, attr_key)
);
-- 예: ('ERP', 'sla_tier', 'gold')
--     ('ERP', 'business_criticality', 'high')
--     ('ERP', 'data_classification', 'internal')
```

5.2 회사·부서·사용자

```
TB_COMPANY (
  company_cd   VARCHAR(10) PK,
  company_nm   VARCHAR(200)
);

TB_DEPT (
  company_cd   VARCHAR(10),
  dept_id     VARCHAR(50),
  dept_cd     VARCHAR(50),
  dept_nm     VARCHAR(200),
  PRIMARY KEY (company_cd, dept_id)
);

TB_USER (
  user_id     VARCHAR(50) PK,
  company_cd   VARCHAR(10) NOT NULL,
  dept_id     VARCHAR(50) NOT NULL,
  user_nm     VARCHAR(100),
  email       VARCHAR(200),
  status      CHAR(1)
);
CREATE INDEX idx_user_dept ON TB_USER(company_cd, dept_id);
```

5.3 메뉴 트리 (Folder/Menu/Link 통합)

```
TB_MENU (
  menu_id         BIGINT      PK,
  system_cd       VARCHAR(20)  NOT NULL,
  parent_menu_id   BIGINT,                      -- 트리 부모 (NULL=루트)

  -- 노드 유형 (핵심)
  menu_type       CHAR(1)     NOT NULL,         -- F:Folder, M:Menu, L:Link

  -- 기본 정보
  menu_cd         VARCHAR(50),                 -- 시스템 내 고유 코드 (선택)
  menu_nm         VARCHAR(200) NOT NULL,
  menu_nm_en       VARCHAR(200),
  menu_desc       VARCHAR(500),
  icon           VARCHAR(50),

  -- 정렬·표시
  sort_order       INT         DEFAULT 0,
  is_visible       CHAR(1)     DEFAULT 'Y',
  is_default       CHAR(1)     DEFAULT 'N',       -- 시스템 진입 시 기본 메뉴

  -- 라이프사이클
  status          CHAR(1)     DEFAULT 'A',
  effective_from   DATE,
  effective_to     DATE,

  created_by, created_at, updated_by, updated_at
);
CREATE INDEX idx_menu_parent ON TB_MENU(parent_menu_id);
CREATE INDEX idx_menu_system ON TB_MENU(system_cd, status);
CREATE INDEX idx_menu_type   ON TB_MENU(system_cd, menu_type);
CREATE UNIQUE INDEX uk_menu_cd ON TB_MENU(system_cd, menu_cd)
  WHERE menu_cd IS NOT NULL;
```

제약 규칙 (애플리케이션 또는 트리거로 강제): - `menu_type='F'`인 메뉴에는 액션 정의 불가 (TB_MENU_ACTION에 row 없음) - `menu_type='F'` 메뉴에는 TB_MENU_IMPL row 없음 - `menu_type='L'` 메뉴는 TB_MENU_IMPL의 external_url 필수 - 폴더의 자식은 폴더 또는 리프 둘 다 가능 - 리프와 링크는 자식을 가질 수 없음

5.4 메뉴 구현체 메타

```
TB_MENU_IMPL (
  menu_id          BIGINT PK,                    -- TB_MENU와 1:1 (M, L 유형)

  -- 라우팅 (프레임워크 무관 공통)
  route_path       VARCHAR(500),                 -- /purchase/requests
  route_name       VARCHAR(100),                 -- PurchaseRequests

  -- SPA 컴포넌트
  component_name    VARCHAR(200),                 -- PurchaseRequestList
  component_path    VARCHAR(500),                 -- @/views/purchase/RequestList.vue

  -- 라우트 메타 (JSON)
  route_meta       JSONB,
  -- 예: { "requiresAuth": true, "layout": "default", "keepAlive": true, "title": "구매요청" }

  -- 라우트 파라미터/쿼리 정의
  route_params     JSONB,                        -- { "id": "string" }
  route_query      JSONB,                        -- { "tab": "string", "filter": "string" }

  -- 외부 링크 (menu_type='L')
  external_url     VARCHAR(1000),
  open_target      VARCHAR(20),                  -- _self/_blank

  -- 화면 옵션
  has_layout       CHAR(1) DEFAULT 'Y',
  is_full_screen    CHAR(1) DEFAULT 'N',
  is_modal         CHAR(1) DEFAULT 'N',

  -- 모바일 대응
  mobile_supported  CHAR(1) DEFAULT 'Y',
  mobile_route_path VARCHAR(500),

  updated_at       TIMESTAMP,
  updated_by       VARCHAR(50)
);
```

5.5 액션·API·매핑

```
-- 액션 마스터 (시스템별)
TB_ACTION (
  system_cd     VARCHAR(20),
  action_cd     VARCHAR(10),
  action_nm     VARCHAR(50),
  sort_order    INT,
  PRIMARY KEY (system_cd, action_cd)
);

-- 메뉴별 사용 가능 액션 (리프 메뉴만)
TB_MENU_ACTION (
  menu_id       BIGINT,
  action_cd     VARCHAR(10),
  PRIMARY KEY (menu_id, action_cd)
  -- 제약: menu_id는 menu_type='M'인 메뉴만 허용
);

-- 메뉴-액션-API 매핑
TB_MENU_ACTION_API (
  menu_id       BIGINT,
  action_cd     VARCHAR(10),
  api_id        BIGINT,
  PRIMARY KEY (menu_id, action_cd, api_id)
);
CREATE INDEX idx_maa_api ON TB_MENU_ACTION_API(api_id);

-- API (샤딩 메타 포함)
TB_API (
  api_id        BIGINT      PK,
  system_cd     VARCHAR(20)  NOT NULL,
  http_method   VARCHAR(10)  NOT NULL,
  url_pattern   VARCHAR(500) NOT NULL,
  url_depth     INT         NOT NULL,    -- 샤딩용
  shard_seg     VARCHAR(64),             -- 샤딩용
  service_nm    VARCHAR(100),
  description   VARCHAR(500),
  status        CHAR(1)
);
CREATE UNIQUE INDEX uk_api_route ON TB_API(system_cd, http_method, url_pattern);
CREATE INDEX idx_api_shard ON TB_API(system_cd, http_method, url_depth, shard_seg);
```

5.6 시스템별 샤딩 컨피그

```
TB_SYSTEM_SHARD_CONFIG (
  system_cd          VARCHAR(20) PK,
  shard_strategy      VARCHAR(20) NOT NULL,
    -- METHOD_DEPTH       : (method, depth)
    -- METHOD_DEPTH_SEG   : (method, depth, seg)
  segment_position    INT,                       -- 0-indexed
  segment_max_length   INT DEFAULT 32,
  segment_fallback    VARCHAR(50) DEFAULT '_root',
  updated_at         TIMESTAMP,
  updated_by         VARCHAR(50)
);
```

5.7 그룹 테이블

```
TB_COMPANY_GROUP (
  company_group_id BIGSERIAL PK,
  group_nm        VARCHAR(100),
  group_type      VARCHAR(20)
);
TB_COMPANY_GROUP_MAP (
  company_group_id BIGINT,
  company_cd      VARCHAR(10),
  PRIMARY KEY (company_group_id, company_cd)
);

TB_DEPT_GROUP (
  dept_group_id    BIGSERIAL PK,
  company_cd      VARCHAR(10),
  group_nm        VARCHAR(100)
);
TB_DEPT_GROUP_MAP (
  dept_group_id    BIGINT,
  company_cd      VARCHAR(10),
  dept_id         VARCHAR(50),
  PRIMARY KEY (dept_group_id, company_cd, dept_id)
);
CREATE INDEX idx_dgmap_dept ON TB_DEPT_GROUP_MAP(company_cd, dept_id);

TB_USER_GROUP (
  user_group_id    BIGSERIAL PK,
  company_cd      VARCHAR(10),
  group_nm        VARCHAR(100),
  group_type      VARCHAR(20)
);
TB_USER_GROUP_MAP (
  user_group_id    BIGINT,
  user_id         VARCHAR(50),
  PRIMARY KEY (user_group_id, user_id)
);
CREATE INDEX idx_ugmap_user ON TB_USER_GROUP_MAP(user_id);
```

v2.2 이전 대비 제거: `TB_MENU_GROUP`, `TB_MENU_GROUP_MAP` (메뉴 트리의 폴더가 그룹 역할을 대체)

5.8 권한 원본 테이블

```
TB_PERMISSION (
  perm_id      BIGSERIAL PK,
  system_cd    VARCHAR(20)  NOT NULL,
  company_cd   VARCHAR(10)  NOT NULL,
  subject_type  VARCHAR(2)   NOT NULL,   -- C/CG/D/DG/U/UG
  subject_id   VARCHAR(50)  NOT NULL,
  target_type   CHAR(1)     NOT NULL,   -- M (메뉴 트리 노드: 폴더 또는 리프)
  target_id    BIGINT      NOT NULL,   -- menu_id
  action_cd    VARCHAR(10)  NOT NULL,
  valid_from   TIMESTAMP,
  valid_to     TIMESTAMP,
  created_by   VARCHAR(50),
  created_at   TIMESTAMP,
  updated_by   VARCHAR(50),
  updated_at   TIMESTAMP
);

CREATE UNIQUE INDEX uk_perm ON TB_PERMISSION
  (system_cd, company_cd, subject_type, subject_id, target_type, target_id, action_cd);
CREATE INDEX idx_perm_subject ON TB_PERMISSION(system_cd, subject_type, subject_id);
CREATE INDEX idx_perm_target  ON TB_PERMISSION(target_type, target_id);
```

v2.2 이전 대비 변경: `target_type` 에서 'G' (MenuGroup) 제거. 폴더(menu_type='F') 메뉴를 부여하면 자손 리프로 자동 전개.

5.9 변경 이벤트 Outbox

```
TB_PERM_CHANGE_LOG (
  seq         BIGSERIAL PK,
  event_type   VARCHAR(30),
  scope_type   VARCHAR(2),
  scope_id     VARCHAR(50),
  system_cd    VARCHAR(20),
  payload      JSONB,
  processed_yn CHAR(1) DEFAULT 'N',
  created_at   TIMESTAMP,
  processed_at TIMESTAMP
);
CREATE INDEX idx_log_unproc ON TB_PERM_CHANGE_LOG(processed_yn, seq)
  WHERE processed_yn = 'N';
```

---

6. 캐시 전략 (샤딩 키 구조)

6.1 설계 원칙

- **L1(Caffeine)과 L2(Redis)는 동일 key 구조**
- 값은 양쪽 모두 **Set**
- 큰 Set을 **(method, depth, [seg])** 으로 샤드 분할

- 샤딩 전략은 **시스템별 컨피그**로 변경 가능
- 폴더 권한은 flatten 단계에서 자손 리프로 전개되어 캐시 적재

6.2 샤딩 전략

6.2.1 전략 1: METHOD_DEPTH (기본)

```
shard = "{method}:{depth}"

키 예시:
perm:api:ERP:U:U12345:GET:3
perm:api:ERP:U:U12345:GET:4
perm:api:ERP:U:U12345:POST:3
```

6.2.2 전략 2: METHOD_DEPTH_SEG (정밀)

```
shard = "{method}:{depth}:{seg}"

seg = URL 분할 후 segment_position 위치의 세그먼트

키 예시 (segment_position=1):
perm:api:ERP:U:U12345:GET:3:purchase
perm:api:ERP:U:U12345:GET:4:purchase
perm:api:ERP:U:U12345:POST:3:order
```

6.2.3 SEG 추출 알고리즘

```
def extract_seg(url_pattern, config):
    if config.shard_strategy != "METHOD_DEPTH_SEG":
        return None

    segments = [s for s in url_pattern.split('/') if s]
    pos = config.segment_position
    if pos >= len(segments):
        return config.segment_fallback

    seg = segments[pos]
    if seg.startswith('{') and seg.endswith('}'):
        return f"_var_{pos}"
    if len(seg) > config.segment_max_length:
        seg = seg[:config.segment_max_length]
    return seg
```

6.2.4 전략 선택 가이드

상황	권장 전략
시스템 도입 초기	METHOD_DEPTH
URL 컨벤션 명확 (`/api/{domain}/...`)	METHOD_DEPTH_SEG (position=1)
사용자 권한 평균 < 100	METHOD_DEPTH
사용자 권한 평균 > 1000	METHOD_DEPTH_SEG
URL 패턴 일관성 없음	METHOD_DEPTH

6.3 통합 키 구조

6.3.1 API 권한 키

```
perm:api:{system_cd}:{level}:{level_id...}:{shard}

level    = C / D / U
level_id = level에 따라 다름
shard    = 시스템 컨피그에 따라
```

Set 멤버: `api_id` 문자열

6.3.2 메뉴-액션 권한 키

```
perm:menu_action:{system_cd}:{level}:{level_id...}:{action_cd}

Set 멤버: menu_id (리프 메뉴만 - 'M' 유형)
```

UI 렌더링에서 메뉴 트리 표시 시 활용. 폴더(F) 메뉴 표시 여부는 자손 리프 중 하나라도 권한이 있으면 표시.

6.3.3 메타 캐시

```
meta:user:{user_id}                  Hash  { company_cd, dept_id }

meta:api:route:{system_cd}           Hash
  field: "{method}:{url_pattern}"
  value: "{api_id}|{depth}|{seg}"

meta:menu_action:{system_cd}:{menu_id}  Hash  { action_cd → [api_id 목록] }

meta:menu_tree:{system_cd}:{menu_id}     Hash
  field: descendants → [자손 리프 menu_id 목록] (폴더 권한 전개 가속)

meta:shard_config:{system_cd}        Hash
  field: strategy           → "METHOD_DEPTH" | "METHOD_DEPTH_SEG"
  field: segment_position   → "1"
  field: segment_max_length → "32"
  field: segment_fallback   → "_root"
```

6.3.4 Warm-up 상태

```
warmup:user:{user_id}:{system_cd}    String "1"  (TTL 1시간)
```

6.4 L1/L2 일관성

```
Caffeine L1                          Redis L2
─────────────────────────────────────────────────────────────────────────────
Key:   "perm:api:ERP:U:U1:GET:4"     Key:   "perm:api:ERP:U:U1:GET:4"
Value: HashSet<String>               Value: Redis Set<String>
      { "1002", "1016", "2034" }    { "1002", "1016", "2034" }
```

6.5 TTL 정책

키 유형	TTL	비고
perm:api:* (C 레벨)	무기한	변경 시 직접 갱신
perm:api:* (D 레벨)	무기한	변경 시 직접 갱신
perm:api:* (U 레벨)	1시간 sliding	활동 사용자만 유지
perm:menu_action:*	동일 패턴	
meta:user:*	1시간	
meta:api:route	24시간	
meta:menu_tree:*	24시간	트리 구조 변경 드묾
meta:shard_config	1시간	
warmup:*	1시간	

6.6 Caffeine 설정

```
Caffeine.newBuilder()
  .maximumWeight(200_000_000L)
  .weigher((key, set) -> ((Set<?>) set).size() * 32 + key.length())
  .expireAfterAccess(Duration.ofMinutes(30))
  .recordStats()
  .build();
```

6.7 캐시 용량 추정

항목	값
user당 평균 샤드 수 (METHOD_DEPTH)	~12개
샤드당 평균 멤버	30~80개
Redis 키 수 (단일 시스템)	~1.24M
Redis 메모리 (단일 시스템)	~600 MB
Caffeine 메모리/서버	100~200 MB

---

7. 권한 조회(인가) 서비스

7.1 API 인터페이스

7.1.1 단일 API 인가 체크

```
GET /api/v1/authz/check
Query: system_cd, company_cd, dept_id, user_id, menu_id, method, api_url
Response: { "allowed": true | false }
```

7.1.2 메뉴 액션 조회

```
GET /api/v1/authz/menu-actions
Query: system_cd, company_cd, dept_id, user_id, menu_id
Response: {
  "allowed_actions": ["R", "C", "U", "A"],
  "ui_hints": { "btn_create": true, "btn_delete": false }
}
```

7.1.3 메뉴 트리 권한 일괄 조회 (UI 렌더링용)

```
POST /api/v1/authz/menu-tree
Body: { "system_cd": "ERP", "company_cd": "...", "dept_id": "...", "user_id": "..." }
Response: {
  "tree": [
    {
       "menu_id": 1,
       "menu_type": "F",
       "menu_nm": "구매관리",
       "icon": "shopping-cart",
       "children": [
        {
            "menu_id": 1024,
            "menu_type": "M",
            "menu_nm": "구매요청",
            "route_path": "/purchase/requests",
            "actions": ["R", "C", "U", "A"]
        }
      ]
    }
  ]
}
```

폴더 메뉴는 자손 리프 중 하나라도 권한이 있을 때만 응답에 포함.

7.1.4 배치 API 체크

```
POST /api/v1/authz/check-batch
Body: { "system_cd": ..., ..., "items": [{ "menu_id", "method", "api_url" }, ...] }
Response: { "results": [{ "allowed": true }, ...] }
```

7.2 조회 처리 로직

```
def check_permission(system, company, dept, user, method, api_url):
    # 1. 메타 해석: api_url → (api_id, depth, seg)
    meta = caffeine.get(f"meta:api:route:{system}",
                    lambda: redis.hgetall(f"meta:api:route:{system}"))
    entry = meta.get(f"{method}:{normalize(api_url)}")
    if not entry:
        return False
    api_id, depth, seg = entry.split("|")

    # 2. 시스템 샤딩 전략
    cfg = caffeine.get(f"meta:shard_config:{system}",
                   lambda: redis.hgetall(f"meta:shard_config:{system}"))

    if cfg["strategy"] == "METHOD_DEPTH_SEG":
        shard = f"{method}:{depth}:{seg}"
    else:
        shard = f"{method}:{depth}"

    # 3. 3개 샤드 키
    keys = [
        f"perm:api:{system}:C:{company}:{shard}",
        f"perm:api:{system}:D:{company}:{dept}:{shard}",
        f"perm:api:{system}:U:{user}:{shard}",
    ]
    target = str(api_id)

    # 4. Caffeine → Redis → contains
    for k in keys:
        s = caffeine.get(k, lambda key: redis.smembers(key) or NEGATIVE_MARKER)
        if target in s:
            return True
    return False
```

7.3 메뉴 트리 응답 처리

UI 사이드바 메뉴 렌더링 시:

```
def get_menu_tree(system, user, company, dept):
    # 1. 시스템 메뉴 트리 구조 (폴더+리프, 캐시됨)
    full_tree = caffeine.get(f"meta:menu_tree:full:{system}", load_full_tree)

    # 2. user의 perm:menu_action Set 병합 (C/D/U 3-way)
    accessible_leaves = get_accessible_leaf_menus(system, company, dept, user)

    # 3. 자손 리프 중 하나라도 권한 있는 폴더만 트리에 포함
    return prune_tree(full_tree, accessible_leaves)
```

7.4 Negative 캐시

- 빈 Set 결과도 짧은 TTL(10초)로 Caffeine 저장
- DB 폭주 방지

---

8. 권한 변경 및 동기화

8.1 변경 이벤트 유형

이벤트 타입	트리거	영향 범위
PERM_GRANT / PERM_REVOKE	TB_PERMISSION 변경	subject별
CG_MEMBER_ADD / DEL	회사그룹 멤버십	추가/제거 회사
DG_MEMBER_ADD / DEL	부서그룹 멤버십	추가/제거 부서
UG_MEMBER_ADD / DEL	사용자그룹 멤버십	추가/제거 사용자
USER_DEPT_CHANGE	사용자 부서 이동	해당 user
MENU_ACTION_API_CHANGE	메뉴-액션-API 매핑 변경	해당 메뉴 참조 전체
MENU_TREE_CHANGE	메뉴 추가/삭제/이동 (폴더 영향)	영향받는 자손 권한 전체
MENU_TYPE_CHANGE	menu_type 변경 (드묾)	해당 메뉴 + 자손
SHARD_STRATEGY_CHANGE	시스템 샤딩 전략 변경	해당 시스템 전체
SEGMENT_POSITION_CHANGE	segment_position 변경	해당 시스템 전체

8.2 영향 샤드 결정

```
def affected_shards(api_ids, system):
    cfg = get_shard_config(system)
    apis = db.query("""
       SELECT api_id, http_method, url_depth, shard_seg
         FROM TB_API WHERE api_id IN ...
    """)
    shards = set()
    for api in apis:
        if cfg["strategy"] == "METHOD_DEPTH_SEG":
            shards.add(f"{api.method}:{api.depth}:{api.shard_seg}")
        else:
            shards.add(f"{api.method}:{api.depth}")
    return shards
```

8.3 폴더 권한 → 자손 리프 전개

폴더 메뉴에 부여된 권한은 flatten 시 자손 리프로 재귀 전개:

```
-- 폴더 권한이 자손 리프로 전개되는 CTE
WITH RECURSIVE menu_descendants AS (
  -- 시작점: 권한 부여된 메뉴
  SELECT menu_id, menu_type FROM TB_MENU
   WHERE menu_id = :target_menu_id
  UNION ALL
  -- 자손 메뉴 재귀 (폴더의 경우)
  SELECT m.menu_id, m.menu_type
    FROM TB_MENU m
    JOIN menu_descendants md ON m.parent_menu_id = md.menu_id
   WHERE m.status = 'A'
)
SELECT menu_id
  FROM menu_descendants
 WHERE menu_type = 'M';     -- 리프만 추출 (실제 권한 부여 의미 있음)
```

8.4 Set Rebuild SQL (User 레벨, 폴더 전개 포함)

```
WITH
-- 1. user의 모든 권한 부여 (직접 + UG)
direct_perms AS (
  SELECT target_id AS menu_id, action_cd
    FROM TB_PERMISSION
   WHERE system_cd = :s AND subject_type = 'U' AND subject_id = :user_id
  UNION
  SELECT p.target_id, p.action_cd
    FROM TB_PERMISSION p
    JOIN TB_USER_GROUP_MAP ugm ON ugm.user_group_id = p.subject_id::bigint
   WHERE p.system_cd = :s AND p.subject_type = 'UG' AND ugm.user_id = :user_id
),
-- 2. 부여된 메뉴(폴더 또는 리프)의 자손 리프 전개
expanded_menus AS (
  SELECT DISTINCT exp.menu_id, dp.action_cd
    FROM direct_perms dp
    JOIN LATERAL (
      WITH RECURSIVE tree AS (
        SELECT menu_id, menu_type FROM TB_MENU WHERE menu_id = dp.menu_id
        UNION ALL
        SELECT m.menu_id, m.menu_type
          FROM TB_MENU m JOIN tree t ON m.parent_menu_id = t.menu_id
         WHERE m.status = 'A'
      )
      SELECT menu_id FROM tree WHERE menu_type = 'M'
    ) exp ON TRUE
)
-- 3. 액션-API 매핑 적용
SELECT DISTINCT a.api_id, a.http_method, a.url_depth, a.shard_seg
  FROM expanded_menus em
  JOIN TB_MENU_ACTION_API maa
    ON maa.menu_id = em.menu_id AND maa.action_cd = em.action_cd
  JOIN TB_API a ON a.api_id = maa.api_id AND a.system_cd = :s
 WHERE a.status = 'A';
```

Dept/Company 레벨도 동일 패턴 (CTE 1단계의 subject 조건만 교체).

8.5 Set Rebuild 패턴

```
def rebuild_user_perm(user, system):
    cfg = get_shard_config(system)

    # 1. 폴더 전개 포함 flatten 쿼리
    rows = db.query(USER_FLATTEN_SQL_WITH_TREE_EXPANSION, user, system)

    # 2. 샤드별 그룹화
    shard_members = defaultdict(set)
    for row in rows:
        shard = make_shard(row, cfg)
        shard_members[shard].add(str(row.api_id))

    # 3. Redis Pipeline 갱신
    pipe = redis.pipeline()
    for k in redis.scan_iter(f"perm:api:{system}:U:{user}:*"):
        pipe.delete(k)
    for shard, members in shard_members.items():
        key = f"perm:api:{system}:U:{user}:{shard}"
        pipe.sadd(key, *members)
        pipe.expire(key, 3600)
    pipe.execute()

    # 4. Pub/Sub publish
    redis.publish("cache:invalidate", json.dumps({
        "scope": "U", "system": system, "id": user
    }))
```

8.6 메뉴 트리 변경 처리

메뉴 추가/삭제/이동 시:

```
def handle_menu_tree_change(menu_id, change_type, system):
    # 1. 영향받는 메뉴 식별
    if change_type == "DELETE":
        affected_menus = [menu_id]
    elif change_type == "MOVE":
        # 이동 전후 부모 트리의 자손 모두
        affected_menus = get_descendants(menu_id) + get_old_parent_descendants()
    elif change_type == "ADD":
        affected_menus = [menu_id]

    # 2. 영향받는 권한 부여 식별
    affected_perms = db.query("""
         SELECT DISTINCT system_cd, subject_type, subject_id
           FROM TB_PERMISSION
          WHERE system_cd = :s AND target_id IN :menu_ids
    """)

    # 3. 폴더 권한이면 영향 더 큼: 부모 폴더에 부여된 권한도 영향
    ancestors = get_ancestor_folders(menu_id)
    affected_perms += find_perms_on_ancestors(ancestors)

    # 4. 영향받는 모든 subject 캐시 rebuild
    for subj in affected_perms:
        enqueue_rebuild(subj)
```

8.7 Caffeine 무효화

```
@EventListener
public void onPubSubMessage(String channel, String message) {
    InvalidateMsg msg = parse(message);
    if (msg.keys != null) {
        for (String key : msg.keys) cache.invalidate(key);
    } else {
        String prefix = buildPrefix(msg.scope, msg.system, msg.id);
        cache.asMap().keySet().removeIf(k -> k.startsWith(prefix));
    }
}
```

8.8 규모별 처리 분기

```
영향 Set 수 추정
  ├── ≤ 10       → 동기 처리
  ├─ ≤ 1,000     → Kafka 비동기 큐 + 청크 병렬
  └── > 1,000    → 배치 JOB
```

폴더 메뉴 변경은 자손 수에 따라 영향이 클 수 있어 비동기 권장.

8.9 샤딩/구현체 변경 처리

변경 종류	영향
TB_SYSTEM_SHARD_CONFIG.shard_strategy	시스템 전체 권한 캐시 재구성
TB_SYSTEM_SHARD_CONFIG.segment_position	시스템 전체 권한 캐시 재구성 + TB_API.shard_seg 일괄 재계산
TB_MENU_IMPL 변경	캐시 영향 없음 (라우터에만 반영)
TB_SYSTEM 메타 변경	캐시 영향 없음 (관리·감사 메타)

---

9. 이벤트 기반 Warm-up

9.1 Warm-up 계층

시점	대상	목적
서버 기동	Company Set 전체 preload	공통 기반
시스템 로그인 (SYSTEM_LOGIN)	User + Dept Set + 메뉴 트리	첫 호출부터 hit
메뉴 진입 (MENU_ENTER)	해당 메뉴 API 매핑 + 자식 prefetch	화면 내 가속
세션 갱신	TTL 연장	지속 hit

9.2 Warm-up 구현

```
def warmup_user(system, user):
    # 1. 사용자의 메뉴-액션 권한과 API 권한 모두 로딩
    cfg = get_shard_config(system)
    shards = db.query(
        "SELECT DISTINCT http_method, url_depth, shard_seg FROM ...flatten..."
    )

    # 2. Pipeline으로 모든 샤드 SMEMBERS
    pipe = redis.pipeline()
    keys = [build_user_key(system, user, shard, cfg) for shard in shards]
    for k in keys:
        pipe.smembers(k)

    # 3. 메뉴 트리도 함께 (UI 빠른 렌더링)
    pipe.smembers(f"perm:menu_action:{system}:U:{user}:R")

    results = pipe.execute()

    # 4. Caffeine 채움
    for k, members in zip(keys, results[:-1]):
        caffeine.put(k, set(members))
```

9.3 Warm-up API

```
POST /api/v1/authz/warmup/system
Body: { "system_cd": "ERP", "user_id": "U12345" }
Response: { "loaded": true, "shard_count": 12, "total_apis": 450, "menu_count": 42 }

POST /api/v1/authz/warmup/menu
Body: { "system_cd": "ERP", "user_id": "U12345", "menu_id": 100, "prefetch_children": true }
Response: { "api_count": 15, "child_menus_preloaded": 8 }
```

9.4 BFF 통합

```
[사용자 로그인]
  BFF → Keycloak 인증 완료
   ↓
BFF → Authz Service: POST /warmup/system (비동기)
   ↓
BFF → Authz Service: GET /menu-tree (메뉴 사이드바용)
   ↓
사용자 첫 화면 렌더링 동안 Warm-up 완료
   ↓
첫 API 호출 시 Caffeine/Redis hit
```

9.5 성능 효과

시나리오	Warm-up 없음	Warm-up 있음
로그인 후 첫 API	~10ms	~1ms
메뉴 진입 후 API	첫 호출 ~2ms	~0.2ms
P99 레이턴시	5~10ms	~2ms

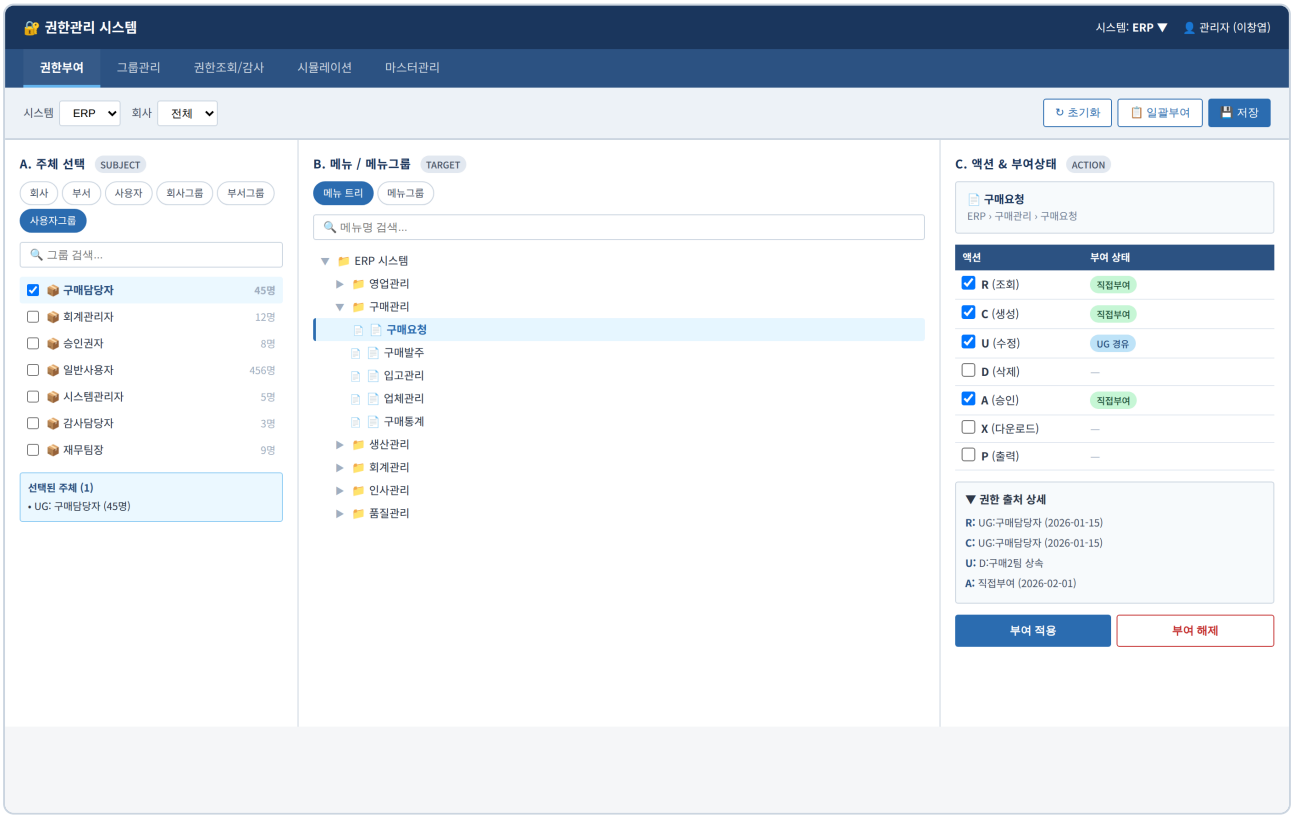
---

10. 관리 화면 UI 설계

10.1 전체 구조

탭	용도	주 사용자
권한부여	일상적 부여/해제	시스템/회사/부서 관리자
그룹관리	CG/DG/UG 멤버십	시스템 관리자
권한조회/감사	역방향 권한 조회	감사담당자
시뮬레이션	변경 전 영향 확인	시스템 관리자
마스터관리	시스템/메뉴/API/액션/샤딩	System Architect

10.2 권한부여 화면 (3-Pane Layout)



좌측 [A] 주체 선택: 6가지 주체 유형(C/D/U/CG/DG/UG) 탭, 검색, 다중 선택

중앙 [B] 메뉴 트리: 시스템별 메뉴 트리 (폴더/리프/링크 시각 구분). **폴더 선택 시 자손 리프 전체에 일괄 부여 효과**

우측 [C] 액션 & 부여상태: - 리프 메뉴 선택 시: 해당 메뉴의 사용 가능 액션 표시 - 폴더 메뉴 선택 시: 자손 리프들의 공통 액션 표시 + 폴더 부여 효과 안내 - 권한 출처 색상 배지 (직접/그룹/부서/회사)

10.3 일괄 부여 모달

🎯 일괄 권한 부여

1 부여 대상 주체

○ 회사 ○ 부서 ○ 사용자 ○ 회사그룹 ○ 부서그룹 ◉ **사용자그룹**

구매담당자 (45명)

ⓘ 멤버 수: **45명** · 소속 회사: 전사 공통

2 부여 대상 메뉴

○ 개별 메뉴 선택 ◉ **메뉴그룹 선택**

구매업무 전체 (12개 메뉴)

포함 메뉴: 구매요청, 구매발주, 입고관리, 업체관리, 구매통계, 계약관리, 단가조회, 재고조회 외 4개

3 부여 액션

✓ **R 조회**	✓ **C 생성**	✓ **U 수정**
☐ D 삭제	✓ **A 승인**	☐ X 다운로드

4 유효 기간 (선택)

☐ 기간 설정 mm/dd/yyyy ~ mm/dd/yyyy

⚠ 영향 범위 사전 확인

부여 건수	**48건**	(메뉴 12 × 액션 4)
영향 사용자		**45명**
신규 허용 API 수		**180개**
캐시 갱신 규모		**perm:api:U:* — 45건 rebuild**
예상 처리 시간		**약 2초**

취소 | 미리보기 | **부여 실행**

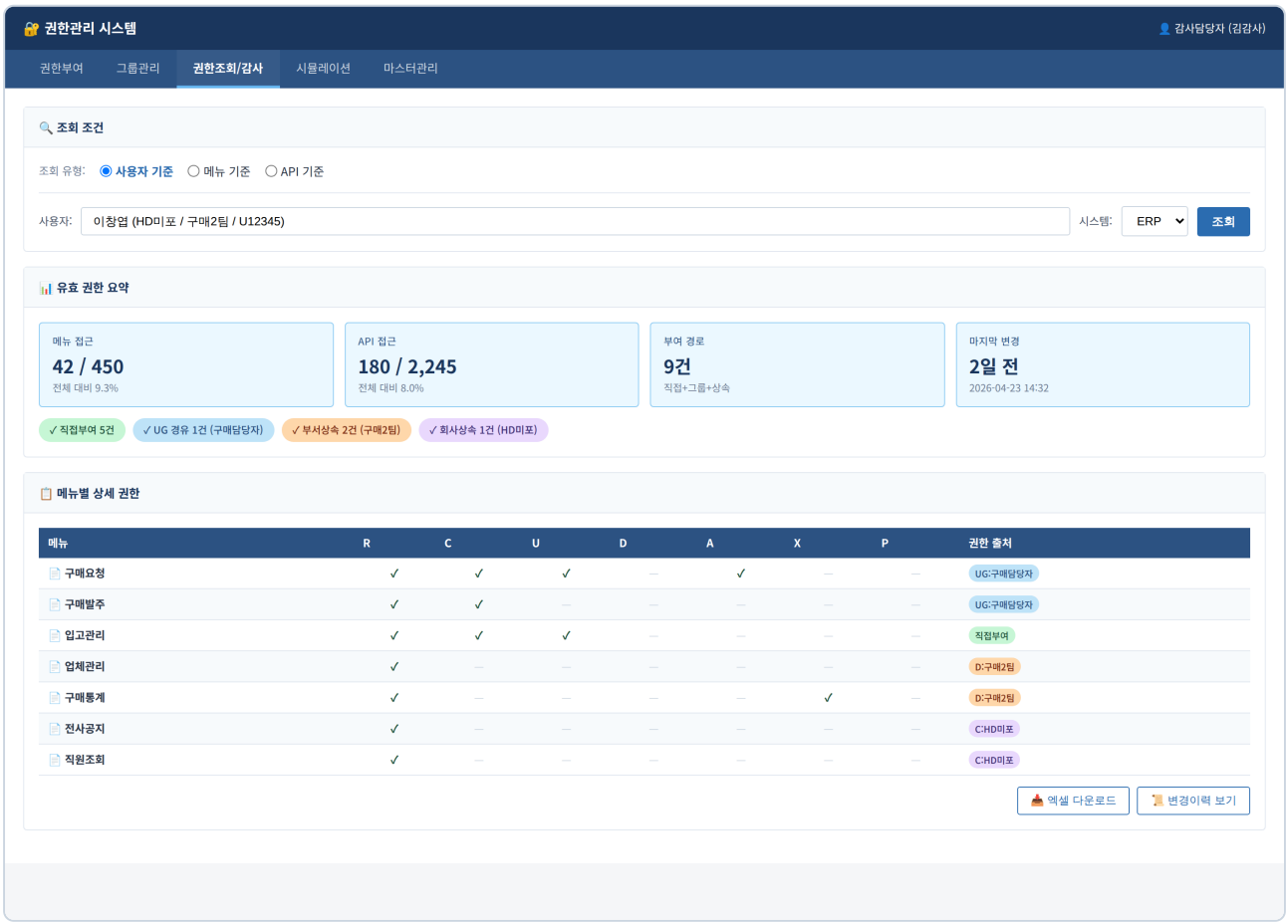
4단계 가이드: 주체 → 메뉴(폴더 가능) → 액션 → 유효기간

폴더 메뉴 선택 시 영향 범위 박스에 "자손 리프 N개" 표시.

10.4 그룹관리 화면

좌측 그룹 목록, 우측 선택 그룹 상세(멤버 / 부여된 권한 / 변경이력 탭). CSV import. 그룹 유형별 상단 탭 (CG/DG/UG).

10.5 권한조회/감사 화면



조회 유형 3가지: 사용자 기준 / 메뉴 기준 / API 기준

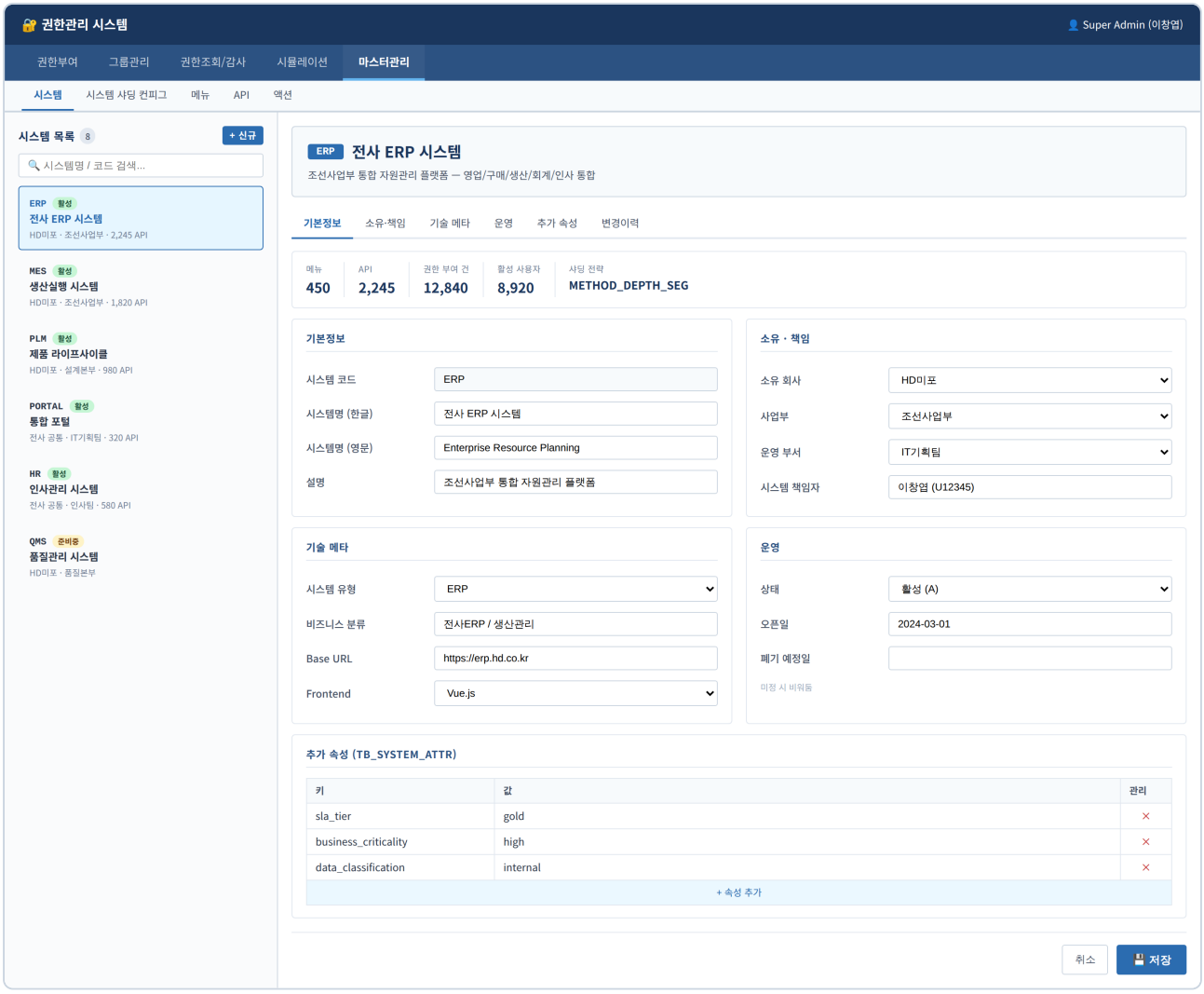
권한 출처에 폴더 메뉴 경유도 표시 (예: "📁 구매관리 폴더 상속").

10.6 시뮬레이션 화면

What-if 분석. 폴더 권한 시뮬레이션 시 자손 리프 영향까지 자동 계산.

10.7 마스터관리: 시스템 관리 화면

시스템 메타정보를 풍부하게 관리.



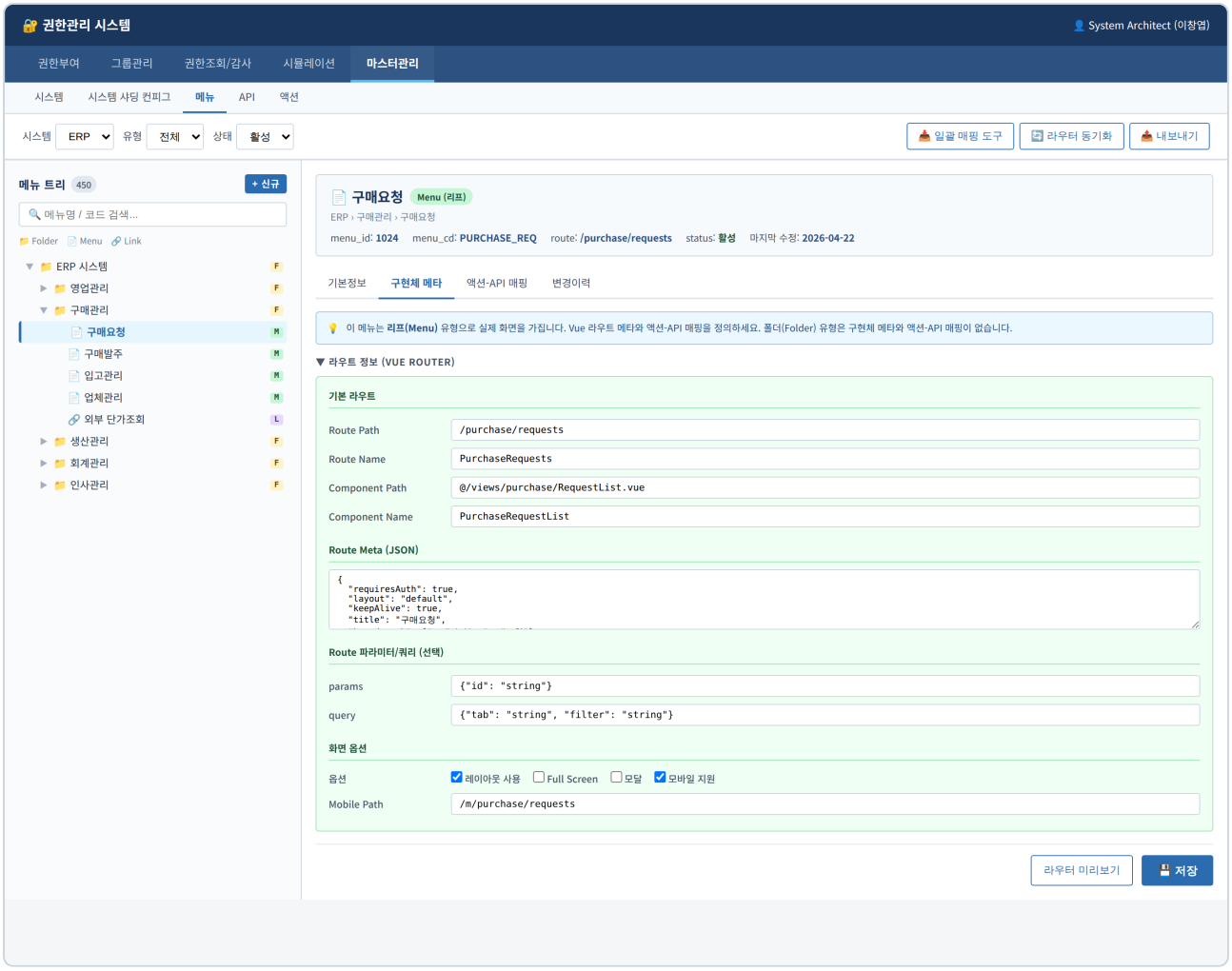
좌측 시스템 목록: 시스템별 코드·소유 회사·사업부·API 수 미리보기

우측 상세 (6 탭): - **기본정보**: 시스템 코드, 한/영 명, 설명 - **소유·책임**: 소유 회사, 사업부, 운영 부서, 시스템 책임자 - **기술 메타**: 시스템 유형(ERP/MES 등), 비즈니스 분류, Base URL, Frontend (Vue/React 등) - **운영**: 상태(A/P/I/D), 오픈일, 폐기 예정일 - **추가 속성**: TB_SYSTEM_ATTR (key-value 자유 확장 — sla_tier, business_criticality 등) - **변경이력**: 메타 변경 추적

상단 통계 바: 메뉴 수, API 수, 권한 부여 건, 활성 사용자, 샤딩 전략

10.8 마스터관리: 메뉴 관리 화면 (핵심)

권한 체계 라이프사이클의 중심. **메뉴 트리 정의 + 구현체 메타 + 액션-API 매핑** 통합.



좌측: 메뉴 트리 - 시스템별 단일 트리 - 노드 유형 시각 구분: 📁 Folder / 📄 Menu / 🔗 Link - 우측에 유형 태그(F/M/L) 표시 - 폴더 펼침/접힘 - 검색 + 신규 메뉴 추가

우측: 메뉴 상세 (4 탭)

[기본정보 탭]

- 메뉴 ID, 코드, 한/영명, 설명, 아이콘, 정렬 순서, 표시 여부, 기본 메뉴 여부, 상태, 유효기간

[구현체 메타 탭] ⭐ 신규

- **Vue Router 정보**: route_path, route_name, component_path, component_name
- **Route Meta (JSON)**: `{ requiresAuth, layout, keepAlive, title, breadcrumb, ... }`
- **Route 파라미터/쿼리**: params, query 정의
- **화면 옵션**: 레이아웃 사용, Full Screen, 모달, 모바일 지원
- **모바일 라우트**: mobile_route_path
- **외부 링크 (Link 유형)**: external_url, open_target
- **라우터 동기화 버튼**: Vue 라우트 정의 파일과 자동 동기화

폴더 유형은 이 탭이 비활성화 (구현체 메타 없음).

[액션-API 매핑 탭]

- 활성 액션 토글 (R/C/U/D/A/X/P)
- 액션별 API 매핑 섹션
- 매핑 0건 액션 ⚠ 경고
- "+ API 추가" 다이얼로그 호출

폴더 유형은 이 탭이 비활성화.

[변경이력 탭]

메뉴 자체 변경 + 구현체 메타 변경 + 액션-API 매핑 변경 통합 표시.

10.8.1 신규 메뉴 추가 다이얼로그

📁 신규 메뉴 추가

부모 메뉴: ERP › 📁 구매관리

메뉴 유형 선택

📁 Folder	📄 Menu	🔗 Link
하위 메뉴를 묶는 중간 노드	실제 화면 (리프) 액션-API 매핑 가능	외부 링크 (다른 시스템 등)

기본정보

메뉴명 (한글)	구매통계
메뉴명 (영문)	Purchase Statistics
메뉴 코드	PURCHASE_STATS 시스템 내 고유 코드 (선택)
아이콘	chart-bar
정렬 순서	5

구현체 메타 (MENU 유형 선택 시)

▼ VUE ROUTER 정보

Route Path	/purchase/statistics
Route Name	PurchaseStatistics
Component Path	@/views/purchase/Statistics.vue
Component Name	PurchaseStatistics

▼ ROUTE META (JSON)

```
{
  "requiresAuth": true,
  "layout": "default",
  "keepAlive": true,
```

화면 옵션 ☑ 레이아웃 사용 ☐ Full Screen ☐ 모달 ☑ 모바일 지원

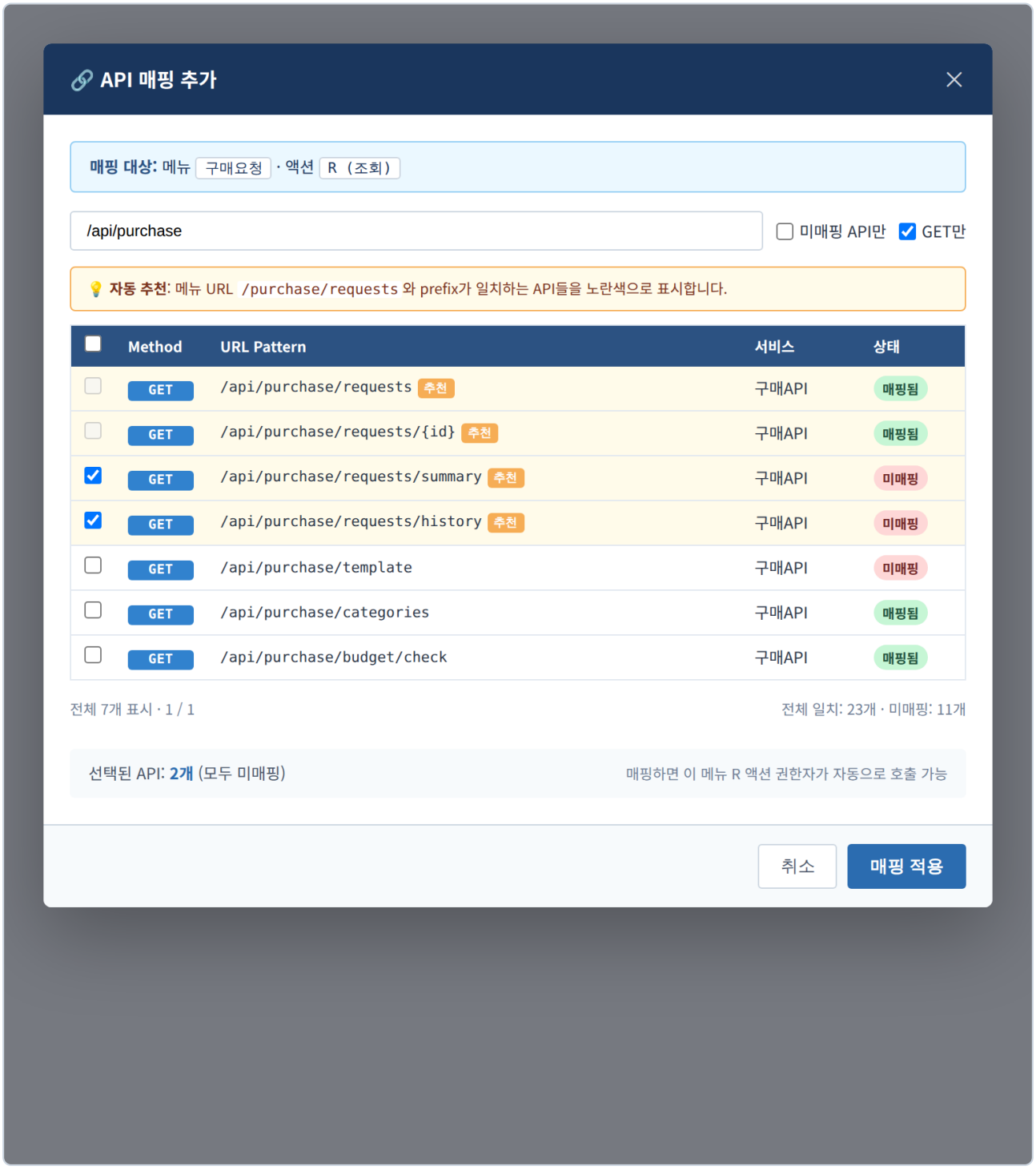
💡 **구현체 메타**는 시스템 운영 중 라우터에 자동 반영됩니다. Vue 라우트 정의 파일과 자동 동기화 가능 (별도 설정).

취소 | 메뉴 생성

3유형 카드 선택: - 📁 **Folder** — 하위 메뉴 묶는 중간 노드 - 📄 **Menu** — 실제 화면 (리프), 액션-API 매핑 가능 - 🔗 **Link** — 외부 링크

기본정보 + 유형별 추가 입력: - Menu 선택 시: Vue Router 정보, Route Meta JSON, 화면 옵션 - Link 선택 시: external_url, open_target - Folder 선택 시: 기본정보만

10.8.2 API 매핑 추가 다이얼로그



- 메뉴 URL과 prefix 일치하는 API 자동 추천 (노란색 강조)
- 미매핑 API 필터
- Method 자동 필터 (액션에 맞게)

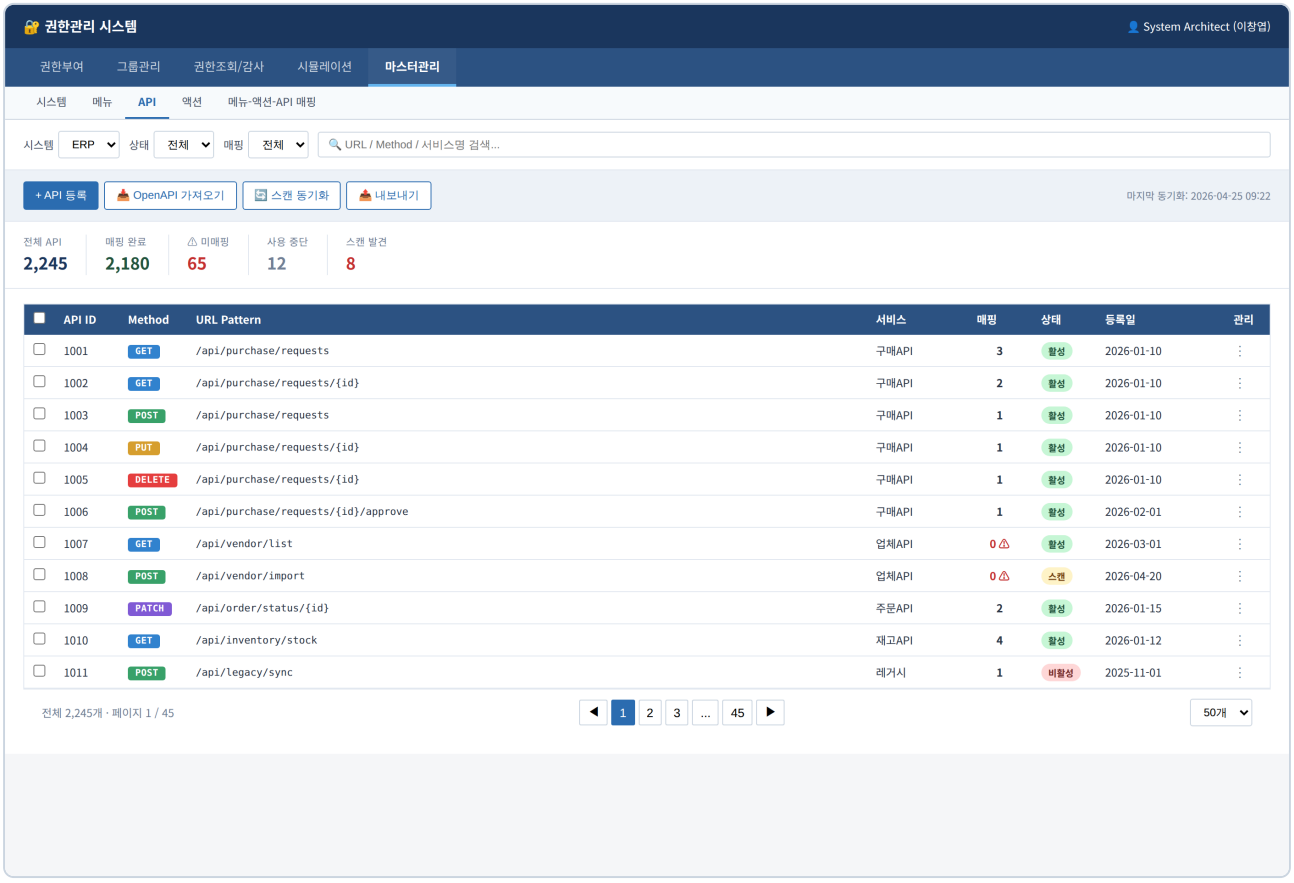
- 이미 매핑된 API 비활성

10.8.3 일괄 매핑 도구

신규 시스템 도입 시 수백~수천 메뉴 일괄 처리: 1. 시스템 선택 2. 자동 추천 알고리즘 실행 (URL prefix 매칭) 3. 추천 결과 검토 4. 일괄 적용 또는 개별 조정

10.9 마스터관리: API 등록·관리 화면

매핑 작업은 메뉴 화면에서 수행. 이 화면은 API 마스터 등록·자동 수집·고아 모니터링 전담.



- 상태 통계 바 (전체/매핑완료/미매핑/사용중단/스캔발견)
- 3가지 등록 방식: 수동, OpenAPI 가져오기, 스캔 동기화
- 매핑 컬럼 read-only (어느 메뉴-액션에서 사용 중)
- Method 색상 배지
- 샤딩 메타 (url_depth, shard_seg) 자동 계산

10.10 마스터관리: 시스템 샤딩 컨피그 화면

```
┌──────────────
```

10.13 관리 화면 자체의 권한 체계

역할	가능 작업
Super Admin	전체 + 마스터관리 + 샤딩 컨피그
System Admin (시스템별)	담당 시스템(소유 회사 기준)의 권한 부여, 메뉴, API 매핑
Company Admin	소속 회사 내 부서/사용자 권한
Dept Admin	소속 부서 내 사용자 권한 (위임)
Auditor	조회/감사 read-only

`TB_SYSTEM.owner_company_cd` 를 활용하여 System Admin이 자동으로 담당 시스템만 보이도록 필터링.

---

11. 성능 목표 및 검증

11.1 목표 대비 달성 평가

지표	목표	예상	달성
평균 응답	< 5ms	~0.3~0.5ms	✓
P95	< 5ms	~1.0ms	✓
P99	< 5ms	~2ms	✓
처리량	1,000 TPS	여유	✓

11.2 샤딩 키 구조의 성능 이점

측정 항목	단일 Set	샤드 Set
단일 Set 평균 멤버 수	500	30~80
Caffeine 엔트리 크기	큼	작음
부분 무효화 정밀도	불가	정밀
Hot/Cold 분리	불가	가능
LRU 효율	낮음	높음
Redis SMEMBERS 시간	~2ms	~0.5ms

11.3 폴더 메뉴 처리 영향

측면	영향
조회 성능	변화 없음 (flatten 단계에서 사전 전개)
변경 성능	폴더 권한 변경 시 자손 수만큼 fan-out 증가
트리 변경	MENU_TREE_CHANGE는 영향 큼 → 비동기 처리 권장

11.4 P99 리스크 및 대응

리스크	대응
Cold start	Warm-up 자동 실행
Redis 네트워크 jitter	로컬 네트워크 배치, Sentinel, 타임아웃
Caffeine GC pause	엔트리 상한, Weight 기반 LRU, G1GC
대량 변경 직후 캐시 미스	Request coalescing, Negative cache
대규모 폴더 권한 변경	비동기 큐, 야간 배치

11.5 모니터링 지표

지표	수집	알람 기준
P50/P95/P99 레이턴시	Prometheus	P99 > 5ms
Caffeine Hit Ratio	Micrometer	< 80%
Redis Hit Ratio	Redis INFO	< 95%
DB Fallback Rate	App 로그	> 5%
Sync Worker Lag	TB_PERM_CHANGE_LOG	> 100건
샤드별 Set 크기 분포	정기 스캔	max > 1000
SEG fallback 비율	정기 스캔	> 30%
폴더 권한 자손 fan-out	변경 이벤트 모니터링	> 10000 user/event

11.6 부하 테스트 시나리오

시나리오	목적
단일 사용자 1,000 TPS 지속	L1 성능 상한
1만 사용자 균등 1,000 TPS	L2 위주 성능
Cold start 1,000 TPS	DB fallback 한계
변경 이벤트 중 1,000 TPS	Rebuild 영향
대규모 폴더 권한 변경	트리 전개 성능
샤딩 전략 전환 중 부하	전환 안정성
Redis Failover	가용성

12. 운영 및 감사

12.1 ITGC 관점 대응

- 모든 권한 변경 → TB_PERM_CHANGE_LOG 완전 기록
- 시스템 메타 변경 → 별도 audit log
- 메뉴 트리/구현체 변경 → 별도 audit log
- 샤딩 컨피그 변경 → 별도 audit log
- 로그인/메뉴 진입 이벤트 → 감사 로그 별도 보관

12.2 백업 및 복구

- DB 정기 백업 (TB_PERMISSION + TB_SYSTEM + TB_MENU + TB_MENU_IMPL + 컨피그 포함)
- Redis 손실 시 DB로부터 전체 rebuild 가능
- Redis 장애 시 DB fallback

12.3 정기 점검

주기	항목
일간	Sync Worker 정상, 미매핑 API 탐지
주간	캐시 Hit Ratio, 레이턴시, 샤드별 Set 크기
월간	고아 API 정리, 미사용 그룹, 샤딩 적정성, 메뉴 구조 점검
분기	권한 재검토 (access review)

12.4 권한 재검토

- 분기별 부서장/그룹장에게 소속원 권한 리스트 발송

- 불필요 권한 회수 요청 수집
- 일괄 해제 도구

12.5 시스템 책임자별 책임 영역

`TB_SYSTEM.owner_user_id`를 활용한 책임 구분: - 시스템 메타 변경 권한 (System Admin 위임) - 시스템별 권한 부여 승인 워크플로우 - 시스템 점검·SLA 보고 책임

12.6 메뉴 구현체 메타 활용

- Vue 프로젝트의 `router/index.js`와 `TB_MENU_IMPL` 동기화 자동화 가능
- 라우터 파일 변경 시 CI/CD 파이프라인에서 TB_MENU_IMPL 업데이트
- 신규 라우트 추가 시 메뉴 자동 등록 후 관리자가 활성화

---

13. 구현 로드맵

13.1 단계별 진행

단계	기간	주요 산출물
Phase 1: 기반	4주	DB 스키마(시스템/메뉴/권한), 기본 API, 단일 시스템
Phase 2: 캐시	4주	Redis 샤드 Set, Caffeine L1/L2, 메타 캐시, 폴더 전개
Phase 3: 동기화	3주	Sync Worker, Outbox, Pub/Sub, 메뉴 트리 변경 처리
Phase 4: 관리UI	7주	권한부여/그룹관리/조회/마스터(시스템·메뉴·API · 샤딩)
Phase 5: 확장	3주	다중 시스템, Warm-up, OpenAPI import, METHOD_DEPTH_SEG
Phase 6: 안정화	4주	부하 테스트, 모니터링, 운영 도구

Phase 4에 1주 추가 (시스템 관리 화면 + 메뉴 구현체 메타 화면).

13.2 우선순위

1. Phase 1-3: 핵심 엔진
2. Phase 4: 운영 가능 수준 UI
3. Phase 5: 다중 시스템 + 정밀 샤딩
4. Phase 6: SLA 확정 및 공식 오픈 전

13.3 기술 스택

영역	선택
애플리케이션	Spring Boot 3.x, MyBatis, MapStruct
데이터베이스	PostgreSQL
캐시	Redis (Sentinel), Caffeine
메시징	Kafka (KRaft), Debezium (옵션)
인증/SSO	Keycloak + OIDC + BFF
API 게이트웨이	Kong 또는 webMethods
모니터링	Prometheus + Grafana + Loki
관리 UI	Vue.js 또는 React

13.4 리스크 및 완화

리스크	영향	완화
대규모 권한 변경 시 캐시 rebuild 지연	일시적 stale	배치 창구, 점진적 rollout
OpenAPI 누락으로 고아 API 발생	권한 부여 누락	CI/CD 통합, 자동 알림
그룹 멤버십 변경 폭증	Sync Worker 지연	파티션 확장, 배치 모드
대규모 폴더 권한 변경	자손 fan-out 큼	비동기, 야간 적용
Redis 장애	성능 저하	Sentinel, DB fallback
샤딩 전략 변경 시 일시적 캐시 미스	응답 지연	야간 적용, 점진 전환
segment_position 부적절	샤딩 효과 감소	정기 모니터링, 위치 재조정
URL 컨벤션 일관성 부족	SEG 추출 실패	METHOD_DEPTH 사용 또는 정규화
메뉴 트리 깊이 과대	폴더 권한 전개 비용	트리 깊이 제한 (예: 5단계)
구현체 메타 동기화 누락	라우팅 불일치	CI/CD 통합, 정기 검증

부록 A: 주요 설계 결정 요약

결정	선택	근거
권한 정책	Allow-only	단순성, 조회 속도
조직 구조	Flat (부서 트리 없음)	요구사항
상속 규칙	Company → User 무조건	요구사항
Flatten 전략	하이브리드 (C/D/U 3-level)	변경 비용 균형
L1/L2 키 구조	통일 (양쪽 동일 key, 값은 Set)	무효화 단순, 메모리 효율
샤딩 차원	(method, depth, [seg])	Set 크기 축소
SEG 추출 방식	명시적 위치 지정 (segment_position)	결정론적, 디버깅 용이
샤딩 컨피그	시스템별 변경 가능	시스템 특성별 최적화
시스템 메타	**소유 회사·사업부·책임자·기술 메타 보유**	운영·감사·책임 명확화
메뉴 구조	**Folder/Menu/Link 단일 트리**	직관적, 그룹 별도 개념 불필요
메뉴그룹	**트리의 폴더로 통합 (TB_MENU_GROUP 제거)**	단순화, 의미 일치
메뉴 구현체 메타	**TB_MENU_IMPL로 분리**	프레임워크 영향 격리, 라우터 동기화
폴더 권한 처리	**flatten 단계에서 자손 리프로 자동 전개**	권한 부여 직관성, 조회 성능 유지
변경 전파	Outbox + Sync Worker + Pub/Sub	정합성, 확장성
시스템 경계	system_cd 필수 prefix	다중 시스템 격리
메뉴-API 연결	메뉴-액션-API 3단 (리프만)	CRUD + 확장 액션
메뉴-API 매핑 UI	메뉴 관리 화면 통합	라이프사이클 일관성
Warm-up	이벤트 기반 (로그인/메뉴진입)	Cold start 제거

부록 B: 용어집

용어	정의
Subject (주체)	권한을 받는 대상 (User/Dept/Company 및 각 그룹)
Target (대상)	권한이 주어지는 자원 (메뉴 트리 노드)
Action	대상에 대한 작업 유형 (R/C/U/D 등)
Folder	메뉴 트리 중간 노드 (menu_type='F')
Menu (Leaf)	실제 화면 메뉴 (menu_type='M', 액션-API 매핑 가능)
Link	외부 링크 메뉴 (menu_type='L')
Implementation Meta	메뉴 구현체 정보 (라우트, 컴포넌트, 화면 옵션)
System Meta	시스템 운영·소유·기술 메타정보
Flatten	상속/그룹/폴더를 해소하여 실효 권한을 평탄화
Fan-out	하나의 변경이 다수 캐시에 파급
Warm-up	조회 전 캐시 선로딩
Outbox	DB 트랜잭션과 이벤트 발행을 정합성 있게 처리
Shard Key	권한 Set을 분할하는 보조 키 차원
Depth	URL 패턴의 디렉토리 세그먼트 수
SEG	URL의 특정 위치 세그먼트 (시스템별 컨피그)
segment_position	0-indexed, URL 분할 후 추출할 인덱스

부록 C: 참고 패턴

- **CQRS**: 조회와 변경 경로 분리
- **Event Sourcing 경량**: Outbox 기반 변경 이벤트
- **Cache-Aside**: 애플리케이션이 캐시 관리 주도
- **Pub/Sub Invalidation**: 분산 캐시 무효화
- **Position-based Sharding**: 명시 위치 기반 결정론적 샤딩
- **Tree-based Permission Inheritance**: 폴더 → 자손 리프 권한 전개

부록 D: 샤딩 키 예시 카탈로그

METHOD_DEPTH 전략

URL	Method	Shard Key
/api/purchase/requests	GET	GET:3
/api/purchase/requests/{id}	GET	GET:4
/api/orders	POST	POST:2

METHOD_DEPTH_SEG 전략 (segment_position=1)

URL	depth	seg	Shard Key
/api/purchase/requests	3	purchase	GET:3:purchase
/api/orders	2	orders	POST:2:orders
/health	1	_root	GET:1:_root

메뉴 구현체 메타 예시 (TB_MENU_IMPL)

```
// 리프 메뉴 (menu_type='M')
{
  "menu_id": 1024,
  "route_path": "/purchase/requests",
  "route_name": "PurchaseRequests",
  "component_path": "@/views/purchase/RequestList.vue",
  "component_name": "PurchaseRequestList",
  "route_meta": {
    "requiresAuth": true,
    "layout": "default",
    "keepAlive": true,
    "title": "구매요청"
  },
  "route_params": { "id": "string" },
  "has_layout": "Y",
  "mobile_supported": "Y",
  "mobile_route_path": "/m/purchase/requests"
}

// 외부 링크 (menu_type='L')
{
  "menu_id": 2050,
  "external_url": "https://price.external-vendor.com",
  "open_target": "_blank"
}
```

시스템 메타 예시 (TB_SYSTEM + TB_SYSTEM_ATTR)

```
TB_SYSTEM:
  system_cd: ERP
  system_nm: 전사 ERP 시스템
  owner_company_cd: HD01 (HD미포)
  owner_division: 조선사업부
  owner_dept_id: D001 (IT기획팀)
  owner_user_id: U12345 (이창엽)
  system_type: ERP
  base_url: https://erp.hd.co.kr
  frontend_type: VUE
  status: A
  go_live_date: 2024-03-01

TB_SYSTEM_ATTR:
  (ERP, sla_tier, gold)
  (ERP, business_criticality, high)
  (ERP, data_classification, internal)
```

문서 끝